

```
In [1]: ! pip install transformers datasets peft
```

Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.47.0)

Requirement already satisfied: datasets in /usr/local/lib/python3.10/dist-packages (3.3.1)

Requirement already satisfied: peft in /usr/local/lib/python3.10/dist-packages (0.14.0)

Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.17.0)

Requirement already satisfied: huggingface-hub<1.0,>=0.24.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.29.0)

Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.2)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.2)

Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.11.6)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)

Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.21.0)

Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.5)

Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.67.1)

Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (19.0.1)

Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (0.3.8)

Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (2.2.3)

Requirement already satisfied: xxhash in /usr/local/lib/python3.10/dist-packages (from datasets) (3.5.0)

Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.10/dist-packages (from datasets) (0.70.16)

Requirement already satisfied: fsspec<=2024.12.0,>=2023.1.0 in /usr/local/lib/python3.10/dist-packages (from fsspec[http]<=2024.12.0,>=2023.1.0->datasets) (2024.12.0)

Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.11.12)

Requirement already satisfied: psutil in /usr/local/lib/python3.10/dist-packages (from peft) (5.9.5)

Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.10/dist-packages (from peft) (2.5.1+cu121)

Requirement already satisfied: accelerate>=0.21.0 in /usr/local/lib/python3.10/dist-packages (from peft) (1.2.1)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (2.4.6)

Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.3.2)

Requirement already satisfied: async-timeout<6.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (5.0.1)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (25.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.5.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (6.1.0)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.

10/dist-packages (from aiohttp->datasets) (0.2.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.18.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.24.0->transformers) (4.12.2)
Requirement already satisfied: mkl_fft in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (1.3.8)
Requirement already satisfied: mkl_random in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (1.2.4)
Requirement already satisfied: mkl_umath in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (0.1.1)
Requirement already satisfied: mkl in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (2025.0.1)
Requirement already satisfied: tbb4py in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (2022.0.0)
Requirement already satisfied: mkl-service in /usr/local/lib/python3.10/dist-packages (from numpy>=1.17->transformers) (2.4.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2025.1.31)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->peft) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->peft) (3.1.4)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->peft) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch>=1.13.0->peft) (1.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2025.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch>=1.13.0->peft) (3.0.2)
Requirement already satisfied: intel-openmp>=2024 in /usr/local/lib/python3.10/dist-packages (from mkl->numpy>=1.17->transformers) (2024.2.0)
Requirement already satisfied: tbb==2022.* in /usr/local/lib/python3.10/dist-packages (from mkl->numpy>=1.17->transformers) (2022.0.0)
Requirement already satisfied: tcmlib==1.* in /usr/local/lib/python3.10/dist-packages (from tbb==2022.*->mkl->numpy>=1.17->transformers) (1.2.0)
Requirement already satisfied: intel-cmplr-lib-rt in /usr/local/lib/python3.10/dist-packages (from mkl_umath->numpy>=1.17->transformers) (2024.2.0)
Requirement already satisfied: intel-cmplr-lib-ur==2024.2.0 in /usr/local/lib/python3.10/dist-packages (from intel-openmp>=2024->mkl->numpy>=1.17->transformers) (2024.2.0)

HW 8: Low Rank Adaptation (LoRA)

In this assignment, you will learn to implement low-rank adaptation both from scratch and using a library—specifically, with PyTorch and the PEFT library, respectively.

This assignment is divided into two sections:

In the first section, we introduce the parameter-efficient transfer learning (PET) method. We use LoRA to adapt the GPT2 model for the SST-2 dataset. This section will teach you how LoRA works and how to implement it from scratch using `forward_hook`.

In the second section, we introduce the PEFT library, which allows us to perform LoRA easily.

Part 1: LoRA from Scratch

With the discovery of scaling properties in deep learning models, several researchers tend to increase model size to achieve emergent properties, especially in the natural language processing (NLP) field. For example, GPT-3 contains 175 billion parameters, making it nearly impossible to fine-tune on limited resources. This trend prevents students like us from adapting these enormous foundation models on a single GPU (or with small resources).

To alleviate this problem, researchers have developed new fine-tuning methods, known as parameter-efficient transfer learning, which allow us to train large models with limited resources. The benefits of these methods extend not only to the training process but also to deployment. After fine-tuning, we only need to save a small number of parameters (the LoRA weights), enabling us to deploy the foundation model to various downstream tasks using minimal storage. One of the prevailing methods is Low Rank Adaptation (LoRA).

Another popular option is prompt tuning, where we only train special tokens that are prepended to the input. However, this is not the focus of this homework.

In this section, we will introduce Low-Rank Adaptation. You are assigned to implement LoRA on GPT2 model. We will finetune the model to SST-2 dataset using the traditional and LoRA method.

Load Dataset and Model

In this step, we will prepare the GPT-2 model and the SST-2 dataset.

SST-2 is a widely used dataset for sentiment analysis, extracted from movie reviews, containing sentences labeled as either positive or negative.

```
In [2]: import torch
import numpy as np
import time
from torch.utils.data import DataLoader
```

```

from transformers import GPT2ForSequenceClassification, GPT2TokenizerFast
from datasets import load_dataset
from tqdm.autonotebook import tqdm
from transformers import GPT2Tokenizer, GPT2ForSequenceClassification, Ad

# Load the GPT-2 model for sequence classification and its tokenizer
model = GPT2ForSequenceClassification.from_pretrained("gpt2", num_labels=
tokenizer = GPT2TokenizerFast.from_pretrained("gpt2")

# GPT-2 does not have a pad token by default so we set it to the EOS token
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

# Load the SST-2 dataset
train_dataset_raw = load_dataset("glue", "sst2", split="train")
train_dataset_raw, val_dataset_raw = train_dataset_raw.train_test_split(t
test_dataset_raw = load_dataset("glue", "sst2", split="validation")

# Preview dataset
print("Sample sentence:")
for data in test_dataset_raw:
    print(data)
    break

def tokenize_function(example):
    return tokenizer(example["sentence"], padding="max_length", truncatio

train_dataset = train_dataset_raw.map(tokenize_function, batched=True)
val_dataset = val_dataset_raw.map(tokenize_function, batched=True)
test_dataset = test_dataset_raw.map(tokenize_function, batched=True)

train_dataset.set_format("torch", columns=["input_ids", "attention_mask",
val_dataset.set_format("torch", columns=["input_ids", "attention_mask", "
test_dataset.set_format("torch", columns=["input_ids", "attention_mask",

# Create data loaders
train_dataloader = DataLoader(train_dataset, shuffle=True, batch_size=16)
val_dataloader = DataLoader(val_dataset, batch_size=16)
test_dataloader = DataLoader(test_dataset, batch_size=16)

```

```

config.json: 0%|          | 0.00/665 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/548M [00:00<?, ?B/s]

```

Some weights of GPT2ForSequenceClassification were not initialized from the model checkpoint at gpt2 and are newly initialized: ['score.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```

tokenizer_config.json: 0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0%|          | 0.00/1.04M [00:00<?, ?B/s]
merges.txt: 0%|          | 0.00/456k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/1.36M [00:00<?, ?B/s]
README.md: 0%|          | 0.00/35.3k [00:00<?, ?B/s]
train-00000-of-00001.parquet: 0%|          | 0.00/3.11M [00:00<?, ?B/s]
validation-00000-of-00001.parquet: 0%|          | 0.00/72.8k [00:00<?, ?B/s]
test-00000-of-00001.parquet: 0%|          | 0.00/148k [00:00<?, ?B/s]
Generating train split: 0%|          | 0/67349 [00:00<?, ? examples/s]
Generating validation split: 0%|          | 0/872 [00:00<?, ? examples/s]
Generating test split: 0%|          | 0/1821 [00:00<?, ? examples/s]

```

Sample sentence:

```
{'sentence': 'it 's a charming and often affecting journey . "', 'label': 1, 'idx': 0}
```

```
Map: 0%|          | 0/53879 [00:00<?, ? examples/s]
```

```
Map: 0%|          | 0/13470 [00:00<?, ? examples/s]
```

```
Map: 0%|          | 0/872 [00:00<?, ? examples/s]
```

Traditional Fine tuning

In the traditional fine-tuning method, the entire model is trained, which is computationally expensive. An alternative approach is to fine-tune only certain layers of the model to reduce resource usage while still adapting the model to a specific task.

To keep the implementation simple, you are assigned to train only the attention weights in the self-attention layers.

The code below displays the names of all layers in the GPT-2 model. This will help you identify which layers to set as trainable or keep frozen. For more details on the attention layers in GPT-2, please refer to the following link: [GPT-2 Attention Layer Details](#).

```
In [3]: for name, module in model.named_modules():  
        print(name, type(module))
```

```

<class 'transformers.models.gpt2.modeling_gpt2.GPT2ForSequenceClassification'>
transformer <class 'transformers.models.gpt2.modeling_gpt2.GPT2Model'>
transformer.wte <class 'torch.nn.modules.sparse.Embedding'>
transformer.wpe <class 'torch.nn.modules.sparse.Embedding'>
transformer.drop <class 'torch.nn.modules.dropout.Dropout'>
transformer.h <class 'torch.nn.modules.container.ModuleList'>
transformer.h.0 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.0.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.0.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SdpaAttention'>
transformer.h.0.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.0.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.0.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.0.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.0.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.0.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.0.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.0.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.0.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.0.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.1 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.1.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.1.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SdpaAttention'>
transformer.h.1.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.1.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.1.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.1.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.1.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.1.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.1.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.1.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.1.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.1.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.2 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.2.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.2.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SdpaAttention'>
transformer.h.2.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.2.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.2.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.2.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.2.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.2.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.2.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.2.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.2.mlp.act <class 'transformers.activations.NewGELUActivation'>

```

```
transformer.h.2.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.3 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.3.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.3.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.3.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.3.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.3.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.3.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.3.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.3.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.3.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.3.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.3.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.3.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.4 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.4.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.4.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.4.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.4.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.4.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.4.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.4.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.4.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.4.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.4.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.4.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.4.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.5 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.5.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.5.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.5.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.5.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.5.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.5.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.5.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.5.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.5.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.5.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.5.mlp.act <class 'transformers.activations.NewGELuActivation'>
transformer.h.5.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.6 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.6.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.6.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.6.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
```



```
transformer.h.6.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.6.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.6.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.6.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.6.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2ML
P'>
transformer.h.6.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.6.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.6.mlp.act <class 'transformers.activations.NewGELUActivatio
n'>
transformer.h.6.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.7 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.7.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.7.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2Sd
paAttention'>
transformer.h.7.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.7.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.7.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.7.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.7.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.7.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2ML
P'>
transformer.h.7.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.7.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.7.mlp.act <class 'transformers.activations.NewGELUActivatio
n'>
transformer.h.7.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.8 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.8.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.8.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2Sd
paAttention'>
transformer.h.8.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.8.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.8.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.8.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.8.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.8.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2ML
P'>
transformer.h.8.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.8.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.8.mlp.act <class 'transformers.activations.NewGELUActivatio
n'>
transformer.h.8.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.9 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.9.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.9.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2Sd
paAttention'>
transformer.h.9.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.9.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.9.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.9.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout
t'>
transformer.h.9.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
```

```

transformer.h.9.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.9.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.9.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.9.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.9.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.10 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.10.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.10.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.10.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.10.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.10.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.10.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.10.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.10.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.10.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.10.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.10.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.10.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.11 <class 'transformers.models.gpt2.modeling_gpt2.GPT2Block'>
transformer.h.11.ln_1 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.11.attn <class 'transformers.models.gpt2.modeling_gpt2.GPT2SelfAttention'>
transformer.h.11.attn.c_attn <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.11.attn.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.11.attn.attn_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.11.attn.resid_dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.h.11.ln_2 <class 'torch.nn.modules.normalization.LayerNorm'>
transformer.h.11.mlp <class 'transformers.models.gpt2.modeling_gpt2.GPT2MLP'>
transformer.h.11.mlp.c_fc <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.11.mlp.c_proj <class 'transformers.pytorch_utils.Conv1D'>
transformer.h.11.mlp.act <class 'transformers.activations.NewGELUActivation'>
transformer.h.11.mlp.dropout <class 'torch.nn.modules.dropout.Dropout'>
transformer.ln_f <class 'torch.nn.modules.normalization.LayerNorm'>
score <class 'torch.nn.modules.linear.Linear'>

```

TODO 1: Freeze the Model and Train Only Attention Weights

You are assigned to freeze the entire model, except for the last two attention weights and the classification head. Note that, in this context, the attention weights do not include the projection layer of the transformer. Instead, they refer only to the weights of the query, key, and value.

HINT: `c_proj` is projection layer.

```
In [4]: for n, p in model.named_parameters():
        # TODO 1: freeze every layer except the last two attention weights and
        # Unfreeze classification head
        if "score" in n or "classifier" in n:
            p.requires_grad = True

        # Unfreeze query, key, value weights of last 2 attention blocks (layer)
        elif (
            ("h.10.attn.c_attn" in n or "h.11.attn.c_attn" in n)
            and "c_proj" not in n
        ):
            p.requires_grad = True

        # Freeze everything else
        else:
            p.requires_grad = False
```

Check Your Answer: The number of learnable parameters is around 3545088.

```
In [5]: pytorch_total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
        print("Number of Trainable Parameters:", pytorch_total_params)
        assert pytorch_total_params == 3545088, "The number of trainable parameters is not 3545088"
```

Number of Trainable Parameters: 3545088

You are assigned to train the GPT-2 model on the SST-2 dataset. Due to the long training time, you will train the model for only 3 epochs. Your model should have around 86-88% accuracy.

```
In [6]: optimizer = AdamW(model.parameters(), lr=5e-5)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
model.to(device)

num_epochs = 3
for epoch in tqdm(range(num_epochs)):
    model.train()
    for batch in tqdm(train_dataloader):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for batch in test_dataloader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["label"].to(device)
            outputs = model(input_ids, attention_mask=attention_mask)
```

```

predictions = torch.argmax(outputs.logits, dim=-1)
correct += (predictions == labels).sum().item()
total += labels.size(0)

accuracy = correct / total
print(f"Epoch {epoch + 1}/{num_epochs} - Accuracy: {accuracy:.4f}")

```

Device: cuda

```

/usr/local/lib/python3.10/dist-packages/transformers/optimization.py:591:
FutureWarning: This implementation of AdamW is deprecated and will be removed in a future version. Use the PyTorch implementation torch.optim.AdamW instead, or set `no_deprecation_warning=True` to disable this warning
warnings.warn(

```

```

0%|          | 0/3 [00:00<?, ?it/s]
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 1/3 - Accuracy: 0.8727
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 2/3 - Accuracy: 0.8750
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 3/3 - Accuracy: 0.8773

```

As you can see, fine-tuning in the traditional way takes a long time to complete and also requires a high-computation GPU to fine-tune the entire model. Therefore, it is not feasible for most people.

In the next part, we will introduce a better method: parameter-efficient learning, which requires lower computation. We will focus on the state-of-the-art method, Low-Rank Adaptation (LoRA).

Low Rank Adaptation

The concept of LoRA is that we are going to estimate the gradient (adaptation matrix) with two smaller matrices (A and B):

$$\text{Adaptation Matrix} = B \times A$$

where $\text{Adaptation Matrix} \in \mathbb{R}^{m \times n}$, $A \in \mathbb{R}^{r \times n}$, and $B \in \mathbb{R}^{m \times r}$. We could make this approximation based on the assumption that Adaptation Matrix has a rank of r . Therefore, the fine-tuned weight becomes:

$$\begin{aligned}
 W &= W_0 + \Delta W \\
 &= W_0 + \frac{\alpha}{r} BA
 \end{aligned}$$

where W denotes the fine-tuned weight, W_0 represents pre-trained weight, ΔW is the gradient and α can be seen as a learning rate. A is initialized using a common initialization, like Kaiming initialization, during the initialization process. On the other hand, B is set to 0 such that the model's output remains the same after injecting LoRA, resulting in a stabilized training process.

To summarize, when injecting LoRA into a layer, we insert new parameters called matrix A and B and initialize them using the above description. Then, we modify the forward pass with `forward_hook` such that the output becomes:

$$h = Wx + \frac{\alpha}{r}BAx$$

where x and h are the input and output, respectively. We recommend you read this [blog](#) to learn more about `forward_hook`.

LoRA on Linear Layer

- TODO 2: initialize A and B to ones (every entry in the matrix is one), such that we can verify your forward pass after attaching the hook.
- TODO 3: implement the forward hook such that new output h is

$$h = Wx + \frac{\alpha}{r}BAx$$

Hint: When you want to declare and initialize a parameter, you can use `torch.nn.Parameter` and `torch.nn.init`, respectively.

```
In [41]: import math
import torch.nn as nn
import torch.nn.functional as F

# Initialize LoRA and attach a hook.
def attach_lora(layer, r, lora_alpha, in_features, out_features):
    assert r > 0, "rank must greater than 0."

    # TODO 2: Initialize A and B to ones
    layer.lora_A = nn.Parameter(torch.ones(r, in_features)) # Shape: (r, in_features)
    layer.lora_B = nn.Parameter(torch.ones(out_features, r)) # Shape: (out_features, r)

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        x = input[0] # Shape: (batch_size, seq_len, in_features)

        # TODO 3: Compute BAx and modify output
        # (B @ A) => (out_features, in_features)
        # (B @ A @ x.T).T = x @ A.T @ B.T
        # Equivalent and easier for batched input:
        delta = (lora_alpha / r) * (x @ layer.lora_A.T @ layer.lora_B.T)

        return output + delta

    return hook
```

To test your `forward_hook`, we will check the difference of the output before and after injecting the LoRA when you initialize matrices A and B with ones.

```
In [42]: from transformers.modeling_utils import Conv1D
# The Conv1D layer from the Transformer library is actually a linear layer

class DummyLinear(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = Conv1D(20, 10)

    def forward(self, x):
        return self.linear(x)
```

```

r, lora_alpha = 1, 4
input_ = torch.arange(10, dtype=torch.float32).unsqueeze(0)
dummy_linear = DummyLinear()
output_before = dummy_linear(input_)
for name, module in dummy_linear.named_modules():
    if isinstance(module, Conv1D):
        in_features, out_features = module.weight.shape
        h = module.register_forward_hook(attach_lora(module, r, lora_alpha, i
output_after = dummy_linear(input_)

if torch.all(torch.isclose(output_after - output_before, lora_alpha * inp
    print("Your forward hook seems to be correct.")
else:
    print("There is something wrong with your forward hook.")

```

Your forward hook seems to be correct.

Instruction

TODO 4: Change the initialization of A and B where A is initialized with Kaiming Uniform ($a = \sqrt{5}$), and B is set to 0.

```

In [43]: import torch.nn.init as init
import torch.nn as nn
import math

def attach_lora(layer, r, lora_alpha, in_features, out_features):
    assert r > 0, "rank must greater than 0."
    # TODO 4: initialize A with kaiming uniform with a = sqrt(5) and init
    layer.lora_A = nn.Parameter(torch.empty(r, in_features))
    layer.lora_B = nn.Parameter(torch.zeros(out_features, r))

    nn.init.kaiming_uniform_(layer.lora_A, a=math.sqrt(5))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # Copy from TODO 3
        lora_update = (layer.lora_B @ layer.lora_A) * (lora_alpha / r)
        output += input[0] @ lora_update.T

    return hook

```

Similar to TODO 1, You are assigned to inject lora into the last two attention weights.

TODO 5: inject lora into the last two attention weights

```

In [48]: r, lora_alpha = 1, 4
def attach_lora_to_maskformer(model, r, lora_alpha):
    hooks = []
    attn_layers = model.transformer.h

    for i in range(len(attn_layers) - 2, len(attn_layers)):
        # TODO 5: inject lora into the last two attention weights
        attn = attn_layers[i].attn
        layer = attn.c_attn
        in_features, out_features = layer.weight.shape

        hook = attach_lora(layer, r, lora_alpha, in_features, out_features)

```

```

hooks.append(layer.register_forward_hook(hook))

return hooks

model = GPT2ForSequenceClassification.from_pretrained("gpt2", num_labels=
model.config.pad_token_id = tokenizer.eos_token_id
hooks = attach_lora_to_maskformer(model, r, lora_alpha)

```

Some weights of GPT2ForSequenceClassification were not initialized from the model checkpoint at gpt2 and are newly initialized: ['score.weight'] You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

TODO 6: Freeze the Model and Train Only LoRA Weights

You are assigned to freeze the entire model, except for the bias of the last two attention weights, LoRA weights, and the classification head.

```

In [49]: for n, p in model.named_parameters():
# TODO 6: freeze every layer except the bias of the last two attention
# Keep LoRA weights trainable
if 'lora_A' in n or 'lora_B' in n:
    p.requires_grad = True

# Keep bias terms of the last two attention layers trainable
elif "transformer.h.10.attn.c_attn.bias" in n or "transformer.h.11.at
    p.requires_grad = True

# Keep classification head trainable
elif "score" in n or "classifier" in n:
    p.requires_grad = True

# Freeze everything else
else:
    p.requires_grad = False

```

Check Your Answer: The number of learnable parameters is around 12288.

```

In [50]: pytorch_total_params = sum(p.numel() for p in model.parameters() if p.req
print(pytorch_total_params)
assert pytorch_total_params == 12288, "The number of trainable parameters
12288

```

```

In [51]: optimizer = AdamW(model.parameters(), lr=5e-5)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
model.to(device)

num_epochs = 3
for epoch in tqdm(range(num_epochs)):
    model.train()
    for batch in tqdm(train_dataloader):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)
        outputs = model(input_ids, attention_mask=attention_mask, labels=

```

```

        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for batch in test_dataloader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["label"].to(device)
            outputs = model(input_ids, attention_mask=attention_mask)
            predictions = torch.argmax(outputs.logits, dim=-1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    accuracy = correct / total
    print(f"Epoch {epoch + 1}/{num_epochs} - Accuracy: {accuracy:.4f}")

```

```

Device: cuda
0%|          | 0/3 [00:00<?, ?it/s]
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 1/3 - Accuracy: 0.7947
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 2/3 - Accuracy: 0.8372
0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 3/3 - Accuracy: 0.8406

```

Part 2: PEFT Library

In the first part, you learned how to implement LoRA from scratch. However, in real-world applications, we can simplify this process by using pre-built libraries. One such library is `peft`, which allows us to inject LoRA into a model more efficiently. By declaring the injected modules in the `LoRAConfig`, we can easily integrate LoRA without having to implement it ourselves. In this section, you will use the `peft` library to apply LoRA to the model.

TODO 7-8: Initialize the LoRA Config and Set trainable parameters

Your task is to initialize `LoRAConfig` using the same hyperparameters as in TODO 6 (`r=1` , `lora_alpha=4`). Apply LoRA only to the last two attention layers. Then, make sure to freeze the entire model except for the LoRA weights, the bias in the LoRA-injected layers, and the classification head. You only need to set the classification head to be trainable where the rest parameters are already set according to our `LoRAConfig` .

HINT: The total number of trainable parameters should match the result from Part 1 (TODO 6).


```
In [60]: from peft import LoraConfig, get_peft_model, TaskType

# TODO 7: Initialize LoRAConfig
lora_config = LoraConfig(
    r=1,
    lora_alpha=4,
    target_modules=[
        f"transformer.h.{model.config.n_layer - 2}.attn.c_attn",
        f"transformer.h.{model.config.n_layer - 1}.attn.c_attn"], # Apply Lo
    modules_to_save=["score"], # Keep classification head t
    bias="lora_only"
)

model = GPT2ForSequenceClassification.from_pretrained("gpt2", num_labels=
model.config.pad_token_id = tokenizer.eos_token_id

model = get_peft_model(model, lora_config)
model = model.to(device)

# TODO 8: Set classification head to trainable
for name, param in model.named_parameters():
    if name.startswith("score"):
        param.requires_grad = True
```

Some weights of GPT2ForSequenceClassification were not initialized from the model checkpoint at gpt2 and are newly initialized: ['score.weight']
 You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
In [61]: pytorch_total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(pytorch_total_params)
assert pytorch_total_params == 12288, "The number of trainable parameters is not 12288"

12288
```

```
In [62]: optimizer = AdamW(model.parameters(), lr=5e-5)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
model.to(device)

num_epochs = 3
for epoch in tqdm(range(num_epochs)):
    model.train()
    for batch in tqdm(train_dataloader):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for batch in test_dataloader:
```

```

input_ids = batch["input_ids"].to(device)
attention_mask = batch["attention_mask"].to(device)
labels = batch["label"].to(device)
outputs = model(input_ids, attention_mask=attention_mask)
predictions = torch.argmax(outputs.logits, dim=-1)
correct += (predictions == labels).sum().item()
total += labels.size(0)

accuracy = correct / total
print(f"Epoch {epoch + 1}/{num_epochs} - Accuracy: {accuracy:.4f}")

```

```

Device: cuda
 0%|          | 0/3 [00:00<?, ?it/s]
 0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 1/3 - Accuracy: 0.8131
 0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 2/3 - Accuracy: 0.8349
 0%|          | 0/3368 [00:00<?, ?it/s]
Epoch 3/3 - Accuracy: 0.8463

```

In []: